

EFFICIENTNET-B0

GÖZ ROI DEEPPFAKE TESPİTİ

Kod Açıklama ve Teknik Mimari Raporu

MODEL	GİRDİ	SINIFLAR
EfficientNet-B0	Combined Eye ROI	Real / Fake

Notebook	efficientnet_b0_eye_baseline_FINAL_STABLE (1).ipynb
Model ailesi	EfficientNet-B0 (torchvision)
Problem tipi	İkili sınıflandırma: gerçek (real) / sahte (fake)
Girdi	224 x 224 piksel combined-eye ROI
Eğitim yaklaşımı	Transfer learning + iki aşamalı eğitim
Rapor amacı	Takım arkadaşlarının kodun mantığını, veri akışını ve eğitim tasarımını kolayca anlayabilmesi

Bu doküman kodu açıklamak içindir; nihai deney performansı run tamamlandıktan sonra sonuç dosyalarından ayrıca raporlanmalıdır.

1. Raporun Amacı ve Deneyin Kısa Özeti

Bu notebook, göz bölgesinden çıkarılmış ROI görüntülerini kullanarak deepfake tespiti yapan bir EfficientNet-B0 tabanlı ikili sınıflandırma deneyi oluşturur. Kod yalnızca model eğitimi değil; veri doğrulama, yerel cache, checkpoint, yeniden başlatma, threshold seçimi, test değerlendirme, grafik üretimi ve inference doğrulaması gibi deneyin tamamını kapsayan uçtan uca bir pipeline şeklinde tasarlanmıştır.

Tek cümlelik özet: Model, iki gözün birlikte bulunduğu combined-eye ROI görüntüsünü alır ve bu görüntünün gerçek bir örneğe mi yoksa deepfake bir örneğe mi ait olduğuna dair fake olasılığı üretir.

1.1 Deneyin Ana Akışı

1	2	3	4	5	6
Metadata şema + split + dosya	Eye ROI combined-eye	Local Cache Colab SSD	EfficientNet transfer learning	Threshold validation	Final Test metrics + figures

1.2 Temel Konfigürasyon

Seed	42
Görüntü boyutu	224 x 224
Batch size	32
DataLoader worker	0
Frozen stage	5 epoch, LR=1e-3
Fine-tuning	15 epoch, LR=2e-5
Optimizer	AdamW
Weight decay	1e-4
Early stopping patience	4
Gradient clipping	1.0
Dropout	0.2
Eğitim hassasiyeti	FP32

2. Veri Yapısı, Girdi ve Sınıflar

Kodun veri kaynağı `metadata.csv` dosyasıdır. Notebook veriyi yeniden rastgele bölmez; metadata içinde daha önceden tanımlanmış `train`, `val` ve `test` splitlerini korur. Bu yaklaşım, farklı modellerin aynı örnekler üzerinde karşılaştırılabilmesini sağlar.

2.1 Kullanılan Sınıflar

Sınıf	Sayısal Etiket	Anlam
real	0	Gerçek / manipüle edilmemiş göz ROI örneği

fake	1	Deepfake / manipüle edilmiş göz ROI örneği
------	---	--

Pozitif sınıf: Kodda `fake` sınıfı pozitif sınıf olarak tanımlanır. Modelin sigmoid çıktısı, örneğin fake olma olasılığı olarak yorumlanır.

2.2 Zorunlu Metadata Alanları

Alan	İşlev	Kontrol
sample_id	Örneğin tekil kimliği	Accepted kayıtlarda boş olamaz ve tekrar edemez
label	real / fake sınıfı	İzin verilen sınıflar dışı değer hata üretir
split	train / val / test	Her split iki sınıfı da içermelidir
status	ROI üretim durumu	Sadece ok/success eğitime adaydır
combined_eye_path	İki gözün birlikte ROI yolu	Dosya fiziksel olarak var olmalıdır

`no\_face` gibi başarısız ROI üretim kayıtları silinmez. Audit amacıyla ayrı tutulur; ancak eğitime dahil edilmez. Bu ayırım, veri kaybını gizlemek yerine neden bazı örneklerin kullanılamadığını izlenebilir kılar.

2.3 Leakage Kontrolü

Kod, aynı resolved eye ROI dosya yolunun train, validation ve test splitleri arasında tekrar etmesini engeller. Buna karşılık mevcut metadata içindeki `video\_id` alanı gerçek kaynak-video kimliği olarak doğrulanmadığı için video-seviyesi leakage kesin olarak doğrulanmış kabul edilmez. Notebook bunu `NOT\_VERIFIABLE\_FROM\_CURRENT\_METADATA` şeklinde raporlar.

3. Ön İşleme ve Veri Yükleme

3.1 Local SSD Cache

Google Drive üzerinden her batch'te dosya okumak eğitim hızını ciddi biçimde düşürebileceği için notebook, uygun ROI görüntülerini `/content/eye\_roi\_cache` dizinine kopyalar. Cache dosyaları isteğe bağlı olarak PIL ile doğrulanır. Eğitim sırasında görüntüler bu yerel kopyalardan okunur.

Neden önemli? Google Drive bir ağ/mount katmanı gibi davranır. Yerel cache, modelin GPU hesaplamasından önce dosya bekleme süresini azaltmayı amaçlar.

3.2 Görüntü Dönüşümleri

Aşama	Train	Validation / Test
Resize	224 x 224, bicubic	224 x 224, bicubic
Horizontal flip	%50 olasılıkla	Yok
Color jitter	Brightness 0.10, Contrast 0.10	Yok
Tensor dönüşümü	Var	Var

Normalizasyon	ImageNet mean/std	ImageNet mean/std
---------------	-------------------	-------------------

Validation ve test setlerinde rastgele augmentation bulunmaz. Böylece değerlendirme aynı örnek üzerinde tekrarlanabilir kalır. ImageNet normalizasyonu kullanılmasının nedeni, backbone'un ImageNet üzerinde önceden eğitilmiş ağırlıklarla başlatılmasıdır.

3.3 Dataset ve DataLoader

`EyeROIDataset` her kayıta RGB görüntüyü açar, transform uygular, etiketi float tensor olarak döndürür ve ayrıca `sample\_id` ile görüntü yolunu taşır. Batch size 32, worker sayısı 0 olarak tanımlanmıştır. Train DataLoader shuffle kullanırken validation ve test shuffle kullanmaz.

4. Model Mimarisi: EfficientNet-B0

Model `torchvision.models.efficientnet\_b0` üzerinden oluşturulur ve `EfficientNet\_B0\_Weights.DEFAULT` ağırlıkları ile ImageNet ön-eğitiminden yararlanır. Böylece model sıfırdan özellik öğrenmek yerine genel görsel özelliklerle başlar ve göz ROI problemine adapte edilir.

4.1 Sınıflandırıcı Başının Değiştirilmesi

Torchvision EfficientNet-B0'in orijinal ImageNet sınıflandırıcı başı kaldırılır. Yerine dropout ve tek çıktı nöronu içeren binary sınıflandırıcı konur:

```
model.classifier = nn.Sequential(
    nn.Dropout(p=0.2),
    nn.Linear(in_features, 1),
)
```

Tek nöron ham `logit` üretir. Bu logit eğitimde `BCEWithLogitsLoss` ile kullanılır. Değerlendirmede `sigmoid(logit)` uygulanarak 0-1 aralığında `P(fake)` olasılığı elde edilir.

4.2 Neden Tek Logit?

- İkili sınıflandırma için iki ayrı softmax çıktısına ihtiyaç duyulmaz.
- `BCEWithLogitsLoss`, sigmoid ve binary cross-entropy hesaplarını sayısal olarak daha güvenli biçimde birleştirir.
- ROC-AUC, PR-AUC ve threshold analizi için doğrudan fake olasılığı üretmek kolaylaşır.

5. Eğitim Stratejisi: İki Aşamalı Transfer Learning

5.1 Stage 1 - Frozen Backbone

İlk aşamada `model.features` parametreleri dondurulur; yalnızca yeni classifier head eğitilir. Konfigürasyona göre bu aşama en fazla 5 epoch ve learning rate 1e-3 ile çalışır.

- Backbone parametrelerinde gradient hesaplanmaz.
- `model.features.eval()` çağrısı ile frozen feature extractor içindeki BatchNorm/Dropout davranışı da sabit tutulur.
- Amaç, rastgele başlatılmış binary sınıflandırıcıyı önceden öğrenilmiş görsel özelliklere adapte etmektir.

5.2 Stage 2 - Full Fine-Tuning

İkinci aşamada en iyi frozen checkpoint yüklenir ve tüm EfficientNet-B0 parametreleri yeniden trainable yapılır. Fine-tuning en fazla 15 epoch, daha düşük learning rate $2e-5$ ile yürütülür.

Mantık: İlk aşama sınıflandırıcıyı stabil hale getirir; ikinci aşama ise backbone filtrelerini göz ROI içindeki deepfake izlerine daha duyarlı olacak şekilde küçük adımlarla günceller.

5.3 Optimizer, Scheduler ve Early Stopping

Optimizer	AdamW
Weight decay	$1e-4$
Scheduler	ReduceLROnPlateau (mode=max, factor=0.5)
Scheduler patience	2
Minimum LR	$1e-7$
Monitör metriği	Öncelikle validation ROC-AUC; AUC hesaplanamazsa validation F1
Early stopping	4 gelişmeyen epoch sonrası
Gradient clipping	max norm = 1.0

Epoch sonunda validation ROC-AUC önceki en iyi değeri aşarsa `best.ckpt` güncellenir ve early-stopping sayacı sıfırlanır. Her tamamlanan epoch ayrıca `last.ckpt` olarak kaydedilir.

6. Sayısal Güvenlik ve Hata Kontrolleri

Notebook eğitim kararlılığı için FP32 kullanır. Her batch'te görüntü, etiket, logit, loss, probability ve gradient değerlerinin finite olup olmadığı kontrol edilir. NaN/Inf tespit edilirse eğitim sessizce devam etmez; hata üreterek durur.

Kontrol	Nerede	Amaç
Input finite	Her batch	Bozuk tensor / preprocessing hatasını yakalamak
Label finite	Her batch	Etiket bozulmasını yakalamak
Logit finite	Forward sonrası	Model sayısal taşmasını yakalamak
Loss finite	Loss sonrası	NaN/Inf kaybı durdurmak
Gradient finite	Backward sonrası	Bozuk güncellemeyi engellemek
Gradient norm	clip_grad_norm_	Patlayan gradientleri sınırlamak
Probability finite	Sigmoid sonrası	Geçersiz tahmini yakalamak

6.1 Smoke Test

Tam eğitim başlamadan önce iki batch üzerinde forward + backward smoke test yapılır. Test için model ağırlıkları geçici olarak güncellenir, ardından yedekten geri yüklenir. Böylece veri, loss ve gradient akışı uzun eğitim başlamadan kontrol edilmiş olur.

7. Checkpoint, Resume ve Reproducibility Tasarımı

Notebook yalnızca ağırlıkları değil; optimizer, scheduler, eğitim geçmişi ve RNG durumlarını da checkpoint içinde saklar. Checkpoint önce geçici dosyaya yazılır, CPU üzerinde tekrar okunarak şema doğrulanır ve ancak daha sonra hedef dosyanın üzerine atomik olarak taşınır.

Bileşen	Kaydediliyor mu?	Neden
Model state	Evet	Ağırlıkları devam ettirmek
Optimizer state	Evet	Momentum/Adam istatistiklerini korumak
Scheduler state	Evet	LR politikasını kaldığı yerden sürdürmek
Epoch + history	Evet	Kaldığı epoch ve metrik geçmişi
Python RNG	Evet	Rastgelelik durumu
NumPy RNG	Evet	NumPy rastgeleliği
Torch CPU RNG	Evet	PyTorch CPU rastgeleliği
CUDA RNG	Evet	GPU rastgeleliği
DataLoader generator	Evet	Train shuffle sırasının devamlılığı

Uyumluluk katmanı: Run ve checkpoint schema sürümleri kontrol edilir. Farklı/bozuk eski checkpointlerin yanlışlıkla yüklenmesi engellenir.

8. Değerlendirme Metrikleri ve Threshold Seçimi

Model tek bir olasılık üretse de sınıf kararı için bir threshold gerekir. Notebook threshold'u test setinden seçmez. Validation setinde 0.05 ile 0.95 arasında tarama yapar ve en iyi F1 değerini veren threshold'u seçer; eşitlik durumunda balanced accuracy ve 0.5'e yakınlık kullanılır.

8.1 Raporlanan Metrikler

Metrik	Ne ölçer?	Projede neden önemli?
Accuracy	Toplam doğru sınıflandırma oranı	Genel performans
Balanced Accuracy	Sınıflara dengeli doğruluk	Sınıf dağılımı etkisini azaltır
Precision	Fake dediğimiz örneklerin ne kadarı gerçekten fake	Yanlış alarm kontrolü
Recall / Sensitivity	Gerçek fake örneklerin ne kadarını bulduk	Deepfake kaçırmama
Specificity	Gerçek örnekleri doğru tanıma oranı	Real örneklerde yanlış pozitifleri görme
F1-score	Precision ve recall harmonik ortalaması	Dengeli sınıflandırma özeti
ROC-AUC	Threshold bağımsız sıralama gücü	Modelin ayırıştırma kapasitesi
Average Precision	PR eğrisi özeti	Pozitif sınıf performansı
TN/FP/FN/TP	Confusion matrix hücreleri	Hata türlerini doğrudan görme

Kritik metodoloji kuralı: Validation threshold se imi ile test de erlendirmesi birbirinden ayrılmıřtır. B ylece test seti model/threshold se imine sızdırılmaz.

9.  retilen Grafikler ve Deney  ıktıları

Dosya / Klas�r	�cerik	Kullanım
checkpoints/frozen	Stage 1 best + last	Frozen e�itimi geri y�kleme
checkpoints/finetune	Stage 2 best + last	Final model
metrics/*history.csv	Epoch bazlı loss ve metrikler	��renme e�rileri
validation_threshold_search.csv	Threshold tarama tablosu	Karar eři�i se�imi
final_test_metrics.csv	Final test metrikleri	Nihai performans
predictions/test_predictions.csv	�rnek bazlı tahminler	Hata analizi
figures	ROC, PR, confusion matrix, curves	Raporlama/g�rselleřtirme
data_accounting.json	Veri muhasebesi	Ka� �rnek kullanıldı/atlandı
run_summary.json	Run �zeti	Tek noktadan deney �zeti
output_manifest.csv	Dosya listesi + hash	�ıktı b�t�nl���
RUN_COMPLETE.json	Run tamamlanma iřareti	Eksik run ile tamamlanmıř run ayrımı

9.1 Grafik Kalite Kapısı

Kaydedilen grafikler 150 DPI ile  retilir ve kısa kenarın en az 600 piksel olması kontrol edilir. Bu kontrol, rapora eklenen g rsellerin okunabilirlik standardını korumak i indir.

10. Inference Reload Quality Gate

Final model yalnızca e itim oturumundaki RAM durumunda test edilmez. Notebook, `best.ckpt` dosyasından sıfırdan yeni bir EfficientNet-B0  rne i oluřturur, checkpoint a rılıklarını y kler ve test batch'inden birkaç  rnek  zerinde inference yapar.

-  ıktı řekli 1 boyutlu olmalıdır.
- Olasılıklar NaN/Inf i ermemelidir.
- T m olasılıklar 0 ile 1 arasında olmalıdır.

Bu kalite kapısı, kaydedilen checkpoint'in ger ekten ba ımsız olarak tekrar y klenebilir ve inference i in kullanılabilir oldu unu do rular.

11. Notebook H crelerinin G revleri

H�cre	Bařlık	G�revi
Cell 1	IMPORTS	PyTorch, torchvision, sklearn, PIL, pandas, matplotlib ve yardımcı k�t�phaneleri y�kler.
Cell 2	CONFIGURATION	T�m deney hiperparametrelerini tek Config sınıfında toplar.
Cell 3	REPRODUCIBILITY	Python/NumPy/Torch seedlerini

		sabitler ve deterministic davranışı yapılandırır.
Cell 4	MOUNT GOOGLE DRIVE	Google Drive'ı Colab ortamına bağlar.
Cell 5	FIXED PROJECT PATHS	Göz ROI input ve Deney 1 / Sonuçlar output yollarını sabitler.
Cell 6	RUN DIRECTORY + SAFE RESUME POLICY	Yeni run klasörü oluşturur veya uyumlu tamamlanmamış run'ı resume eder.
Cell 7	ATOMIC I/O	JSON/CSV/YAML/checkpoint dosyalarını atomik yazan yardımcı fonksiyonları tanımlar.
Cell 8	LOAD + VALIDATE EYE ROI METADATA	Metadata şema, sınıf, split, sample_id, dosya ve leakage kontrollerini yapar.
Cell 9	LOCAL SSD IMAGE CACHE	Eye ROI görüntülerini Colab lokal SSD'ye cache eder.
Cell 10	TRANSFORMS + DATASETS + DATALOADERS	Transform, Dataset ve DataLoader nesnelerini kurar.
Cell 11	5-BATCH I/O SPEED SANITY CHECK	5 batch ile DataLoader hızını ölçer.
Cell 12	MODEL	ImageNet ağırlıklı EfficientNet-B0 modelini ve tek-logit classifier'ı oluşturur.
Cell 13	METRICS	Accuracy, F1, specificity, AUC vb. metrikleri ve threshold taramasını tanımlar.
Cell 14	SAFE CHECKPOINT + RNG HELPERS	Checkpoint uyumluluğu ve RNG save/restore mekanizmasını kurar.
Cell 15	SAFE FP32 EPOCH RUNNER WITH LIVE PROGRESS	FP32 train/eval epoch loop; NaN/Inf ve gradient kontrollerini yapar.
Cell 16	TWO-BATCH MODEL SMOKE TEST	2 batch forward/backward smoke test çalıştırır.
Cell 17	TRAINING FUNCTION	Optimizer, scheduler, checkpoint ve early stopping içeren stage eğitim fonksiyonu.
Cell 18	STAGE 1: FROZEN EFFICIENTNET-B0 BACKBONE	Backbone dondurulmuş Stage 1 eğitimini çalıştırır.
Cell 19	STAGE 2: FULL FP32 FINE-TUNING	Tüm ağı açarak düşük LR ile Stage 2 fine-tuning yapar.
Cell 20	VALIDATION THRESHOLD SELECTION	En iyi fine-tune checkpoint üzerinde validation threshold'u seçer.
Cell 21	FINAL TEST EVALUATION	Seçilen threshold ile final test ve örnek bazlı prediction CSV üretir.
Cell 22	FIGURES	Loss, validation metrics, ROC, PR, confusion matrix ve threshold grafiklerini üretir.
Cell 23	RELOAD / INFERENCE QUALITY GATE	Checkpoint'i yeni modelde yeniden yükleyip inference kalite testi yapar.
Cell 24	FINAL MANIFEST + RUN SUMMARY + COMPLETE MARKER	Manifest, run summary ve COMPLETE işaretini yazar.

12. Kodun Güçlü Yönleri ve Dikkat Edilmesi Gereken Noktalar

12.1 Güçlü Yönler

- Veri splitleri yeniden oluşturulmadığı için deneyler arası karşılaştırma daha tutarlıdır.
- Başarısız ROI kayıtları gizlenmez; audit dosyasına yazılır.
- Validation threshold ile test değerlendirmesi metodolojik olarak ayrılmıştır.
- Checkpoint ve RNG yönetimi, uzun Colab eğitimlerinde kaldığı yerden devam etmeyi destekler.
- Smoke test, finite kontrolleri ve inference reload testi hata yakalama katmanları oluşturur.
- Çıktı klasör yapısı; metrik, tahmin, grafik ve manifestleri ayrı tuttuğu için raporlamayı kolaylaştırır.

12.2 Sınırlılıklar / Teknik Notlar

- Pipeline kare/ROI tabanlıdır; göz kırpmaya, gaze veya kareler arası temporal tutarlılığı doğrudan modellemez.
- Gerçek kaynak-video kimliği metadata ile doğrulanamadığı için video-seviyesi leakage kesin olarak kanıtlanamaz.
- FP32 sayısal olarak güvenli olsa da özellikle full fine-tuning aşamasını yavaşlatır.
- 224 x 224'e resize işlemi, çok küçük fiziksel göz artefaktlarında bilgi kaybına yol açabilir.
- Horizontal flip ve color jitter dışında augmentation çeşitliliği sınırlıdır.
- Nihai performans yorumu için final test metrikleri ve mümkünse farklı dataset/cross-dataset testleri ayrıca incelenmelidir.

13. Takım Arkadaşına 1 Dakikada Nasıl Anlatılır?

Kısa anlatım: Elimizde her kareden çıkarılmış iki gözün birlikte bulunduğu görüntüler var. Bunları real ve fake olarak etiketliyoruz. ImageNet üzerinde önceden eğitilmiş EfficientNet-B0'ı alıp önce sadece son sınıflandırma katmanını eğitiyoruz. Sonra tüm ağı çok düşük learning rate ile fine-tune ediyoruz. Model her ROI için fake olasılığı veriyor. Karar eşliğini testten değil validation setinden seçiyoruz; en son test setinde Accuracy, F1, Recall, Specificity, ROC-AUC ve PR-AUC gibi metrikleri raporluyoruz.

13.1 Akılda Kalması Gereken 8 Nokta

- Girdi: 224 x 224 combined-eye ROI.
- Sınıflar: real=0, fake=1.
- Backbone: EfficientNet-B0 + ImageNet pretrained weights.
- Output: tek logit -> sigmoid -> P(fake).
- Loss: BCEWithLogitsLoss.
- Eğitim: frozen head training + full fine-tuning.
- Threshold: validation setinden seçilir.
- Final test: seçilmiş threshold ile tek sefer değerlendirilir.

14. Sonuç

Bu notebook yalnızca bir EfficientNet-B0 eğitim scripti değildir; veri doğrulamasından final inference testine kadar deney yaşam döngüsünü yöneten bir baseline pipeline'dır. Mimari olarak görece hafif

bir CNN ailesi kullanıldığı için, daha karmaşık Transformer veya hibrit göz modellerinin katkısını ölçmek için anlamlı bir karşılaştırma noktası sağlar.

Takım içi kullanım açısından en önemli kavramlar; combined-eye ROI girdisi, real/fake ikili sınıflandırma, ImageNet transfer learning, iki aşamalı eğitim, validation tabanlı threshold seçimi ve test setinin yalnızca final değerlendirmede kullanılmasıdır.

Nihai raporlama notu: Bu kod açıklama raporunda final deney skoru uydurulmamıştır. Eğitim tamamlandıktan sonra `final\_test\_metrics.csv`, `run\_summary.json` ve `figures/` altındaki grafikler kullanılarak ayrı bir Deney Sonuç Raporu hazırlanmalıdır.